

Méthodes de Clustering. Algorithmes.

Hard clustering: 1 individu appartient à un cluster donné.

Soft clustering: Probabilités d'appartenance possibles.

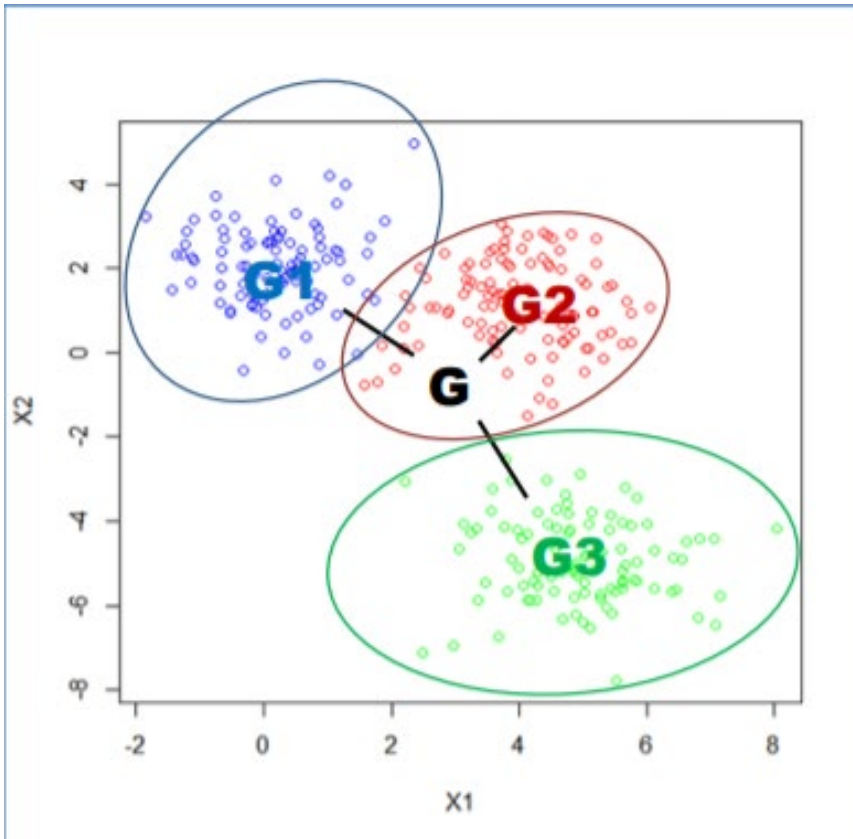
- **Centroid-Based** → **Centres d'inertie des clusters calculés.** **K MEANS**
Le nombre de clusters doit être défini initialement
- **Connectivity-Based** → **distances entre points calculées.** **CAH**
Choix du meilleur nombre de clusters
- **Distribution-Based** → **Distributions statistiques** **Gaussian Mixtures**
Recherche de populations représentatives
- **Density-Based** → **Densités de points recherchées** **DBSCAN**
il n'est pas capable de gérer des clusters de densités différentes

K Means. Les données

- **Variables catégorielles non prises** → transformer en « Facteur »
 - Transformer variables catégorielles en « dummies » avec ***pandas.get_dummies***
 - Transformer les données textuelles (tf-idf)
- **Valeurs manquantes à remplacer** ou ligne éliminée
- **Données centrées réduites** pour réduire la disparité entre variables

K Means: Classification par partition

→ Rôle crucial des centres de classes



Minimiser l'inertie intra-classes W , nombre de classes K fixés.

d = distance qui caractérise les proximités entre individus.

Relation fondamentale (Théorème d'Huygens)

Inertie totale = Inertie inter - classes + Inertie intra - classe

$$T = B + W$$

$$\sum_{i=1}^n d^2(i, G) = \sum_{k=1}^K n_k d^2(G_k, G) + \sum_{k=1}^K \sum_{i=1}^{n_k} d^2(i, G_k)$$

Points rattachés à un groupe selon leur proximité avec le barycentre associé

K Means. Contraintes.

Types de datasets

- Points forts (un des algo de clustering les plus populaire)
 - Facile à implémenter
 - Convient aux gros datasets
 - Très efficace pour les clusters sphériques
 - Rapide
- Points faibles
 - Choix initial du nombre de clusters K
 - Résultats variables sur différentes exécutions d'un algorithme
 - Uniquement pour données numériques

Classification par partition: caractéristiques

- Fixer à priori le nombre de classes K
- Définir une partition de départ
- Réallocation des observations pour obtenir une partition meilleure
- Optimiser une mesure d'évaluation de la partition
- Fournir une partition unique des données

K Means. Validations

- Validation des clusters
 - minimal intra cluster distance
 - maximal inter cluster distance.

Deux types de métriques:

- ***Mesures intra clusters.*** Inertia scores

WCSS

Inertie minimale recherchée

- ***Mesures entre différents clusters.*** Silhouette Coefficient.

Silhouette_score

Score silhouette max (1)

→ meilleur

Score silhouette min (-1)

→ aucune difference entre clusters

Score silhouette 0

→ clusters overlapping

K Means. Librairies.

Scikit-Learn

<https://scikitlearn.org/stable/modules/generated/sklearn.cluster.KMeans.html>

KMeans(n_clusters = X, init=k-means++, n_init=??, max_iter=??, random_state=??)

n_init=25 → meilleur que « random_state » fixé (random_state=None)

1. Evaluer les partitions (propositions de nb de clusters)

from sklearn.cluster import Kmeans

from sklearn import metrics

Score “silhouette” à calculer

Faire varier le nb de clusters de 2 à 10

range_n_cluster = range(2,10)

km = KMeans(n_clusters=k)

km.fit(X)

res.append(metrics.silhouette_score(X,km.labels_))

wcss.append(km.inertia_)

plot nb. clusters vs. silhouette score

import matplotlib.pyplot as plt

plt.plot(np.arange(2,10,1),res)

plt.plot(np.arange(2,10,1),wcss)

plt.show()

K Means. Librairies.

Scikit-Learn

```
KMeans(n_clusters = X, init=k-means++,n_init=??, max_iter=??, random_state=??)
```

```
Km.fit(df) (ou df_ACP)
```

```
Clusters_labels= km.fit_predict(df)
```

```
Pred = pandas.DataFrame(Clusters_labels)
```

```
Pred.columns = [« cluster »]
```

```
df_new = pandas.concat([df,Pred], axis=1)
```

Dataset contenant une colonne cluster identifiant les groupes détectés

Les validations métriques

Index Davies Bouldin

```
from sklearn.metrics import davies_bouldin_score
```

```
davies_bouldin_score(coord, Clusters_labels)
```

Inertie intra/inter

```
from sklearn.metrics import silhouette_score
```

```
score=silhouette_score(coord_Clusters_labels)
```

% de la variance expliquée par le clustering

K Means. Validations

Calcul des moyennes des variables pour le dataset complet

Calcul de $n \times \text{variance}$

Calcul du nombre d'individus dans chaque groupe

Calcul des moyennes des variables dans chaque groupe

Calcul des différences Moy générale-Moy par groupe pour chaque variable

Pondération vs la taille des groupes

Somme= Somme de toutes les différences de Moy pondérées

Calcul du rapport $\text{Somme} / (n \times p)$

→ Obtention du % de variance du dataset expliqué par le clustering

Interprétation des clusters

Entrée [61]:

```
▶ m=df.mean()  
#total sum of squares  
TSS=data.shape[0]*data.var  
print(TSS)  
gb = df.groupby(kmeans.labels_)  
nk=gb.size()  
print(nk)  
#moyenne conditionnelle  
mk=gb.mean()  
print(mk)  
#diff cond.means et overall means  
EMk=(mk-m)**2  
#ponderation taille des groupes  
EM=EMk.multiply(nk,axis=0)  
#sum  
BSS=np.sum(EM,axis=0)  
print(BSS)  
#proportion de variance expliquée  
R2=BSS/TSS  
#print(R2)
```

Datasets exemples

- Classifications données continues
- Classifications données continues + catégorielles
- Classifications données textuelles
 - TF-IDF (pour clustering de documents).
- Déconvolutions d'images